



深度学习与计算机视觉

实战：搭建网络实现图像分类

王一帆

■ 训练集和测试集

训练集：估计模型中的参数，使模型能够预测分类

测试集：评估模型的预测性能

模型在训练集上进行训练，在测试集上评估性能；对于一个训好的模型，训练集数据是它见过的，测试集数据是它没见过的。

■ 实战：在测试集上进行推理预测，得到准确率

目前我们可以输入一张图像并得到分类结果，接下来我们将在测试数据集上遍历所有图片，并计算模型在测试集上的准确率，步骤如下：

1. 构建DataLoader进行数据加载
2. 载入权重
3. 推理预测计算准确率



■ torch.utils.data.DataLoader()

```
DataLoader( dataset,  
            batch_size=1,  
            shuffle=False,  
            drop_last=False)
```

DataLoader的参数有很多，我们主要关注以下参数：

- dataset: Dataset类，决定数据从哪读取以及如何读取
- bathsize: 可选参数，批大小
- shuffle: 可选参数，是否乱序
- drop_last: 可选参数，当样本数不能被batchsize整除时，是否舍弃最后一批数据

■ batch size

在网络训练过程中，每次只送入一张图像效率是比较低的，且对网络训练的性能也有所影响，能不能一次向网络送入多张输入呢？每次批量送入的样本数量大小就是batch size



■ 一些关于训练的术语

- Epoch: 所有训练样本都已输入到模型中, 称为一个Epoch
- Iteration: 一批样本输入到模型中, 称为一个Iteration
- Batchsize: 一批样本的大小, 决定一个Epoch有多少个Iteration

假设样本总数80, Batchsize是8, 那么1Epoch分为10 Iteration。

■ shuffle

在网络训练过程中，我们需要对数据进行多个epoch的训练，如果 `shuffle=True`，每次循环的数据顺序都会被打乱



■ drop_last

假设样本总数87， Batchsize是8， 最后一批数据不够8个，
drop_last=True的情况下将会把这7个数据舍弃



■ dataset

torch.utils.data.DataSet类，需要装载进DataLoader的原始数据集，其一般由三部分组成：

- `__init__`：初始化数据集
- `__len__`：返回数据集的长度
- `__getitem__`：索引数据，在DtaLoader的循环之中会调用这个方法

因此在建立dataset时，要继承DataSet类，如：

```
from torch.utils.data import Dataset
```

```
class MNIST_Dataset(Dataset):
```

```
    def __init__():
```

```
        .....
```

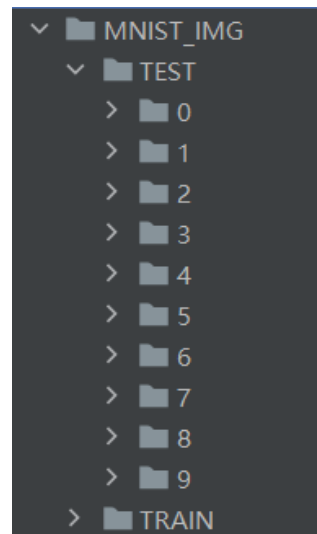
■ dataset

- `__init__`:

初始化列表，用于存储数据集的路径，如：

```
class MNIST_Dataset(Dataset):
    def __init__(self, root, isTrain=True):
        self.dataset = []
        # 判断数据集是训练集还是测试集
        if isTrain:
            folder = "TRAIN"
        else:
            folder = "TEST"
        # 利用os模块，用字符串拼接图片的路径
        for tag in os.listdir(root + "/" + folder):
            for file in os.listdir(root + "/" + folder + "/" + tag):
                path = root + "/" + folder + "/" + tag + "/" + file
                # 存储图片的路径和标签
                self.dataset.append((path, tag))
```

目录结构：



上面这段代码，先初始化一个dataset列表，然后用`os.listdir()`函数遍历path中的所有文件，再把文件路径放入dataset列表中



■ dataset

- `__len__` :

返回数据集的长度，如：

```
def __len__(self):  
    # 返回数据集的长度  
    return len(self.dataset)
```

■ dataset

- `__getitem__` :

在DataLoader循环时调用此方法，处理图片，在这里我们需要根据`__init__`中初始化的dataset列表读取图片和标签，并对图像做预处理，如：

```
def __getitem__(self, index):  
    data = self.dataset[index]  
    # 获取图像和标签的路径  
    path = data[0]  
    tag = data[1]  
    img = cv2.imread(path, 0)  
    # 图像预处理  
    img = img.reshape(-1)  
    img = img / 255  
    return np.float32(img), int(tag)
```



■ dataset

dataset建立好后，调用DataLoader

```
dataset_test = MNIST_Dataset("MNIST_IMG", isTrain=False)
dataloader_test = DataLoader(dataset=dataset_test,
                              batch_size=100,
                              shuffle=True)
```

■ 载入权重

新建网络，载入训练好的模型权重

```
net = MNIST_Net()  
net.load_state_dict(torch.load('ckpt_4layers.pt', map_location='cpu'))
```



■ 进入DataLoader循环

进入DataLoader的循环，遍历测试集进行测试：

```
for i, (img, label) in enumerate(data_loader_test):
```

- i: 索引
- (img, label): 建立dataset时__getitem__()的返回值

■ 预测推理

和推理一张图像类似，用网络模型进行预测推理

```
for i, (img, label) in enumerate(data_loader_test):  
    # 开始测试模式  
    net.eval()  
    # 把数据加载在网络中  
    test_out = net(img)  
    # 10分类，各自概率，取最大值的索引  
    softmax = nn.Softmax(dim=1)  
    test_out = softmax(test_out)  
    pre = torch.argmax(test_out, dim=1)
```

■ 计算得分

对于手写数字的十分类任务，可以用准确率Accuracy进行评估

$$Accuracy = \frac{\text{正确分类样本数}}{\text{所有样本数}}$$

```
score = torch.mean(torch.eq(pre, label).float())
```

- torch.eq(input1, input2)

判断input1和input2两个张量是否相同，相同的位置返回True，反之返回False；返回值是和input相同大小的张量



■ 实战：训练MNIST分类模型

1. 构建DataLoader进行数据加载
2. 构建网络
3. 设置优化器和损失函数
4. 训练网络并测试模型性能
5. 保存模型



■ dataset

和测试时构建的dataset类似，但注意我们要遍历训练集数据

```
self.dataset_train = MNIST_Dataset("MNIST_IMG", isTrain=True)
self.data_loader_train = DataLoader(dataset=self.dataset_train,
                                     batch_size=600,
                                     shuffle=True)
```

■ 构建网络

调用之前定义好的网络结构

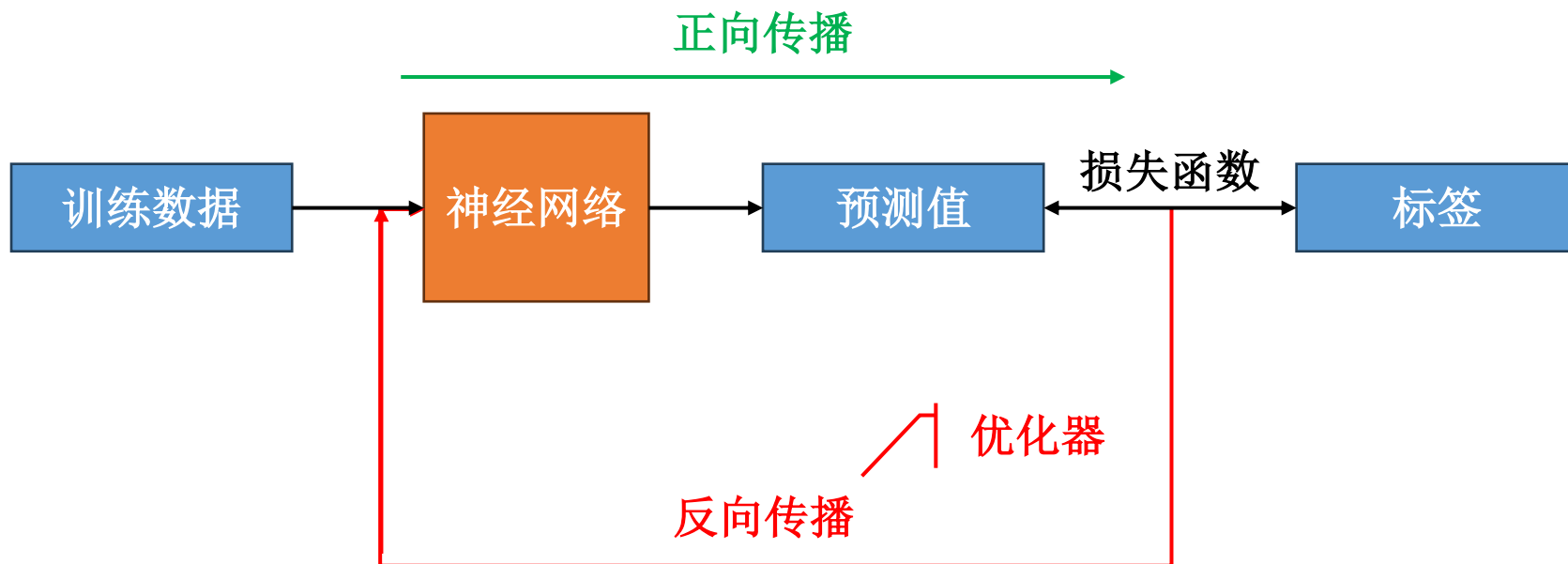
```
self.net = MNIST_Net()

self.layer1 = nn.Sequential(
    nn.Linear(784, 512),
    nn.ReLU()
)
self.layer2 = nn.Sequential(
    nn.Linear(512, 256),
    nn.ReLU()
)
self.layer3 = nn.Sequential(
    nn.Linear(256, 64),
    nn.ReLU()
)
self.layer4 = nn.Linear(64, 10)
```

```
# 进行前向传播返回结果
def forward(self, x):
    x = self.layer1(x)
    x = self.layer2(x)
    x = self.layer3(x)
    x = self.layer4(x)
    return x
```

■ 损失函数

每一个样本经过模型后会得到一个预测值，得到的预测值和真实值的差值就成为损失（损失值越小证明模型越准确）；有许多不同种类的损失函数，这些函数本质上就是**计算预测值和真实值的差距的一类型函数**





■ 交叉熵损失函数 (Cross Entropy Loss)

对单个样本，假设真实分布为 y ，网络输出分布为 y' ，总的类别数为 n ，则在这种情况下，交叉熵损失函数的计算方法为：

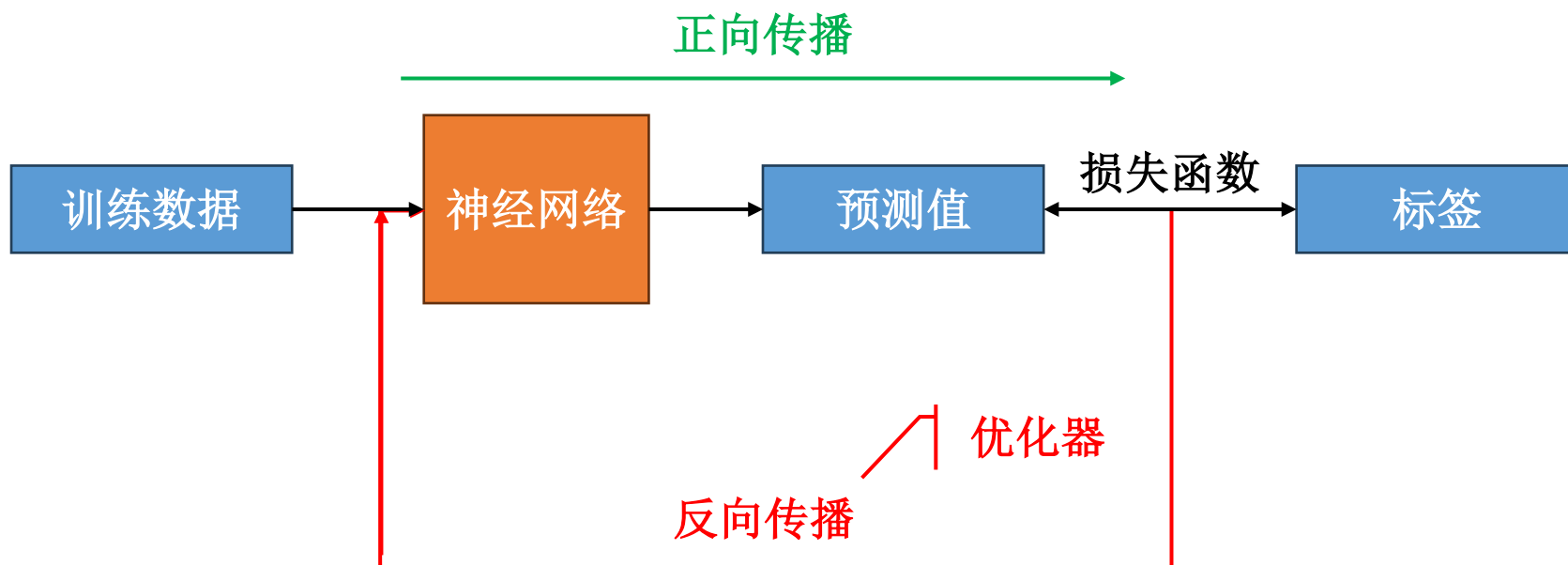
$$Loss = - \sum_{i=1}^n y_i \log \hat{y}_i'$$

pytorch已经帮我们封装好了交叉熵损失函数：

`nn.CrossEntropyLoss()`

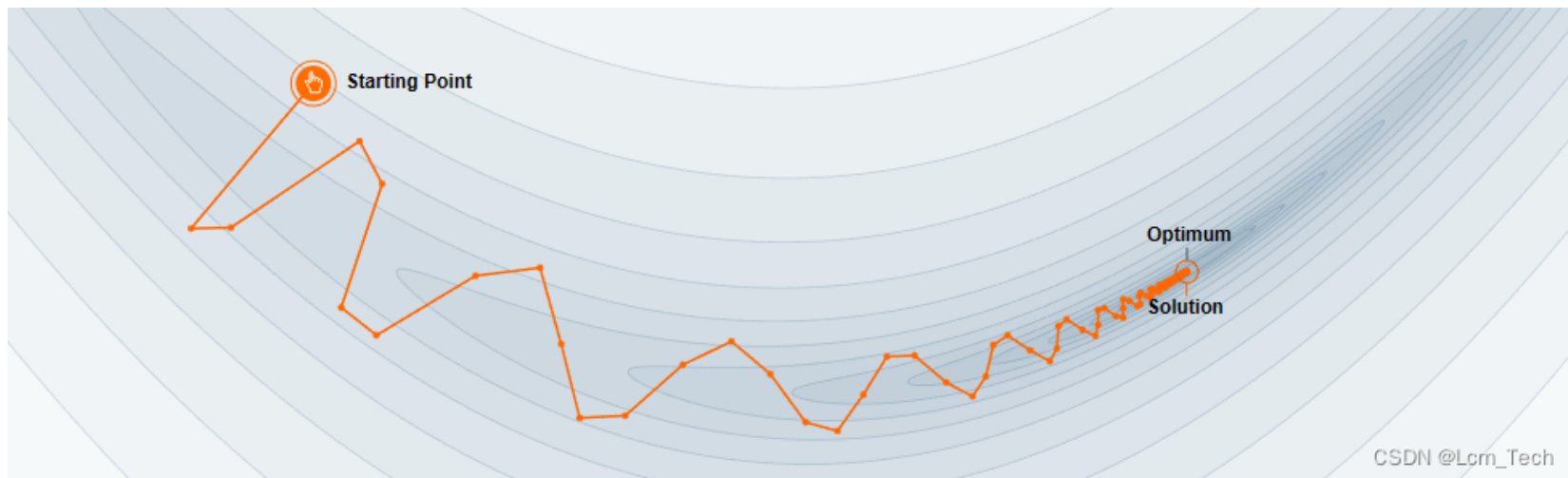
■ 优化器

管理并更新模型中可学习参数的值，使得模型输出更接近真实标签。通俗一点，就是采样梯度更新模型的可学习参数，使得损失减小。



■ 梯度下降

网络通过求损失函数的梯度更新网络参数，使得损失值最小（即输出和标签越来越相似）





■ SGD——随机梯度下降法

`optim.SGD(params, lr, momentum=0)`

首先优化器需要我们导入`optim`

`from torch import optim`

- `params`: 管理的参数组, 即我们网络的参数`net.parameters()`
- `lr`: 学习率, 代表优化器更新参数的步幅, 学习率过大, 容易错过收敛点; 学习率过小, 收敛速度慢
- `momentum`: 动量系数, 在当前`step`参数更新中加入了部分上一个`step`的梯度



■ 设置优化器和损失函数

```
# 创建模型优化器，后面的梯度清空，更新参数都由这个优化器自动完成
self.opt = optim.SGD(self.net.parameters(),
                      lr=0.01,
                      momentum=0.9)

# 求损失：交叉熵损失函数
self.loss = nn.CrossEntropyLoss()
```

■ 训练网络并测试模型性能

训练网络的每次循环都在重复：

1. 数据输入网络，得到输出
2. 计算损失
3. 梯度更新

每个epoch训练结束后，在测试集上测试一下模型的精确度

■ 数据输入网络，得到输出

self.net.train()开启训练模式，启用 batch normalization

```
for i, (img, label) in enumerate(self.dataloader_train):  
    # 开启训练模式  
    self.net.train()  
    # 前向传播结果  
    out = self.net(img)
```



■ 计算损失

前面我们已经定义了`self.loss=nn.CrossEntropyLoss()`，这里我们直接调用计算`out`和`label`的损失。

这里`out`是一个向量，大小为`[1, 10]`，`label`是数字索引，即`0~9`

```
loss = self.loss(out, label)
```



■ 梯度更新

- `self.opt.zero_grad()`: 清空所管理参数的梯度, pytorch特性: 张量梯度不自动清零, 会将张量梯度累加; 因此, 需要在用完梯度之后, 或者在反向传播前, 将梯度自动清零
- `loss.backward()`: 反向传播
- `self.opt.step()`: 执行一步更新

```
self.opt.zero_grad()  
loss.backward()  
self.opt.step()
```

■ 保存模型

`torch.save(params, path)`

- `params`: 要保存的模型参数
- `path`: 保存路径

```
torch.save(self.net.state_dict(), 'ckpt.pt')
```